

SURE ProEd

Full Stack Development

Objective

To understand how AI tools can support learning in the React internship while mentors continue to act as Subject Matter Experts (SMEs).

Module 1 — Web & JavaScript Foundations

 Objective: Learn basics of web development so students can build simple webpages & use AI for doubts, reducing mentor teaching time.



Students Learn

- How websites work (client, server, browser)
- HTML tags, elements, forms, layout
- CSS – Flexbox, Grid, media queries
- JavaScript:
 - Variables, arrays, objects
 - Functions & events
 - Loops, conditions
 - DOM manipulation
 - Fetch API (GET + POST)



How AI Helps

- Explain JS & CSS concepts (chat-based learning)
- Ask AI to fix errors & bugs
- Auto-generate HTML/CSS templates
- Create practice questions
- Convert designs → HTML/CSS



Mentor Focus

- Real coding habits (indentation, naming, file structure)
- How to think in steps before coding
- Debugging mindset (break problem → find cause → fix)
- Teaching Git basics: commits, push, pull

AI Tools & Platforms

- Primary: ChatGPT for conceptual learning, Cursor AI for code completion
- Code Generation: GitHub Copilot for HTML/CSS templates
- UI Prototyping: v0.dev or Bolt.new for rapid HTML/CSS generation

Documentation Resources

- MDN Web Docs:
- developer.mozilla.org/en-US/docs/Web/JavaScript
- - Comprehensive JS reference
- MDN CSS:
- developer.mozilla.org/en-US/docs/Web/CSS
- - Complete CSS guide
- JavaScript.info: Modern JavaScript tutorial with interactive examples
- Web.dev: Google's web development best practices

Interactive Tools

- jsv9000.app: JavaScript event loop visualizer (already mentioned)
- JavaScript Visualizer: pythontutor.com/javascript.html - Step through code execution
- Flexbox Froggy: flexboxfroggy.com - Learn Flexbox through games
- Grid Garden: cssgridgarden.com - CSS Grid interactive tutorial
- DOM Visualizer: Interactive DOM tree visualization tool
- Chrome DevTools: Built-in debugging and inspection

Video Resources

- FreeCodeCamp: "JavaScript Full Course" on YouTube
- Traversy Media: "HTML & CSS Crash Course"
- The Net Ninja: "JavaScript DOM Tutorial" series
- Web Dev Simplified: Modern JavaScript concepts

Module 2 — React.js Development

 Objective: Learn how to turn UI into reusable components & make dynamic frontend apps.

Students Learn

- SPA concept — how React works
- JSX, Components, Props, State
- Lists, Keys, Events, Conditional rendering
- useState, useEffect
- Fetching data from API inside React
- Routing (react-router-dom)
- Basic UI using Tailwind / Bootstrap
- Folder structure in React

How AI Helps

- Generate React components quickly
- Ask "why component not re-rendering?"
- Convert plain JavaScript → React logic
- Get reusable code & examples
- Debug errors instantly

Mentor Focus

- Component architecture – breaking UI properly
- Real-time debugging walkthrough
- Reusable component patterns
- Folder naming standards
- How to handle frontend performance

AI Tools & Platforms

- Best for UI: Perplexity Pro - Superior for explaining component architecture and UI patterns
- Best for Code: Claude (Anthropic) - Excellent at React component logic and hooks
- Design-to-Code: Builder.io Visual Copilot - Convert Figma designs to React components
- VS Code Extension: Sapling - Visualize React component tree

Documentation Resources

- React Official Docs: react.dev - New interactive documentation
- React TypeScript Cheatsheet: react-typescript-cheatsheet.netlify.app
- Patterns.dev: Design patterns for React applications
- React Router Docs: reactrouter.com/en/main

Interactive Tools

- React DevTools: Browser extension for component inspection
- React Sight: Live component hierarchy visualization
- CodeSandbox: codesandbox.io - Online React playground
- StackBlitz: stackblitz.com - Instant React development environment
- React Flow: reactflow.dev - Build node-based diagrams showing component relationships

Video Resources

- Codevolution: "React Tutorial for Beginners" (comprehensive series)
- Web Dev Simplified: "Learn React Hooks" series
- FreeCodeCamp: "React Course - Beginner's Tutorial"
- Net Ninja: "Full Modern React Tutorial"

Module 3 — Backend (Node.js + Express.js)

 Objective: Learn how server works, create APIs, send/receive data, and build backend logic.

Students Learn

- Node.js basics – runtime, npm, package.json
- Express.js routes, controllers
- REST APIs – GET, POST, PUT, DELETE
- JSON request-response cycle
- Middleware basics (logging, parsing)
- Error handling
- Connecting backend → MongoDB

How AI Helps

- Ask for backend architecture templates
- Request complete CRUD API boilerplate
- Debug "Cannot GET /api" type errors
- Write documentation examples
- Learn backend patterns fast

Mentor Focus

- API design standards
- Validation structure
- Proper MVC folder setup
- Error handling the professional way
- How backend connects to frontend step-by-step

AI Tools & Platforms

- Best for Backend: Gemini Advanced or Claude - Superior for complex server architecture and API design
- Best for Debugging: Phind - Developer-focused AI search engine
- Code Completion: Tabnine - Excellent context-aware Node.js suggestions
- API Testing: GitHub Copilot - Generate test cases and API endpoints

Documentation Resources

- Node.js Official: nodejs.org/docs - Core Node.js documentation
- Express.js Guide: expressjs.com/en/guide - Complete Express reference
- MDN HTTP: developer.mozilla.org/en-US/docs/Web/HTTP
- REST API Design: github.com/dwyl/learn-api-design
- API Best Practices: restfulapi.net - RESTful conventions

Interactive Tools

- Postman: API testing and documentation platform
- Thunder Client: VS Code extension for API testing
- Insomnia: REST client with GraphQL support
- RequestBin: webhook.site - Test webhooks and HTTP requests
- Mermaid Live: mermaid.live - Create API flow diagrams
- Swagger Editor: editor.swagger.io - API documentation design

Video Resources

- FreeCodeCamp: "CRUD API Tutorial - Node, Express, MongoDB"
- Traversy Media: "Express JS Crash Course"
- Programming with Mosh: "Node.js Tutorial for Beginners"
- WsCube Tech: "Backend Development Full Course - 6 Hours"
- Djamware: "Build RESTful API 2025 Edition"

Module 4 — MongoDB & Data Layer

 Objective: Store & retrieve data properly and understand schema design.

Students Learn

- MongoDB basics: collections, documents
- Schema design (Mongoose)
- Relationships (1-to-many, ref + populate)
- Querying (find, update, delete)
- Index basics
- Environment variables (.env)

How AI Helps

- Ask AI to generate schemas & validation rules
- Get sample queries for practice
- Ask "why data not saving?" & debug

Mentor Focus

- Choosing correct schema structure for scalability
- Avoiding common DB mistakes
- Teaching indexing & performance concepts
- Reviewing student schema & giving corrections

AI Tools & Platforms

- Schema Design: ChatGPT or Claude - Excellent for explaining schema patterns
- Query Building: GitHub Copilot - Generate Mongoose queries
- Optimization: Gemini - Better at explaining performance concepts
- Documentation: Perplexity - Quick access to MongoDB docs with citations

Documentation Resources

- MongoDB Manual: docs.mongodb.com/manual - Complete reference
- Mongoose Docs: mongoosejs.com/docs - Schema and query guide
- MongoDB University: learn.mongodb.com
- Database Design: MongoDB data modeling patterns
- MongoDB Blog: mongodb.com/blog - Best practices and updates

Interactive Tools

- MongoDB Compass: Official GUI for MongoDB
- MongoDB Atlas: Cloud-hosted MongoDB with free tier
- Mongo Playground: mongoplayground.net - Test queries online
- Moon Modeler: datenssen.com - Visual schema designer
- Mermaid ER Diagrams: Create entity-relationship diagrams

Video Resources

- MongoDB University: Free official courses
- Net Ninja: "MongoDB Tutorial for Beginners"
- FreeCodeCamp: "MongoDB Crash Course"
- Academind: "MongoDB Complete Guide"
- WsCube Tech: MongoDB chapters in backend course

AI Tool Strategy: When to Use What

ChatGPT

Best for: Quick concept explanations, JavaScript fundamentals, general debugging, learning syntax Limitations: May provide outdated library versions, less specialized for specific frameworks

Perplexity

Best for: UI/UX generation, React component architecture, getting cited sources, research-based queries Why: Better at providing current documentation references and explaining visual concepts

Claude (Anthropic)

Best for: Complex React logic, refactoring code, explaining design patterns, full-file code generation Why: Superior context window and code reasoning abilities

Gemini Advanced

Best for: Backend architecture, database optimization, complex business logic, multi-step algorithms Why: Strong at system design and explaining performance implications

GitHub Copilot

Best for: Real-time code completion, boilerplate generation, test writing Why: IDE integration and context from your codebase